

> Embedded system

① Super loop system

```

void main()
{
    while(1)
    {
        Read Btn;
        Display LCD; → wait(1)
        Read Sensor; → wait(2)
        Led on;
        Motor on;
        Led off;
    }
}

```

Adv

Determinism → High

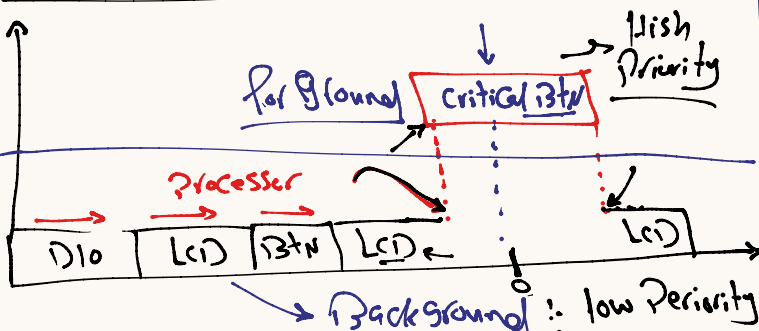
dis

→ low Responsiveness

Btn

LCD ADC
0s 1ms 2ms 3s

② Por/Back Ground system



→ Based on → Interrupt → Processor

Adv

* High Responsiveness

dis

* Low Determinism

super loop

- * High Determinism
- * Low Responsiveness
- * Based on while(1)

Por/Back Ground

- * Low Determinism
- * High Responsiveness
- * Based on
 - ↳ while(1) ∴ Back
 - ↳ Interrupt ∴ Por

③ Real time operating system (RTOS)

↳ High Determinism ↳ High Responsiveness

* Interrupt

① Processor will be Interrupted

② Internal Processor
↳ outside Processor

Another Peripherals

ADC
timer
uart
SPI
I2C

outside mc

↳ signal
↳ Btn
↳ sensor Display

Interrupt ∴ AVR → Vectored Interrupt

∴ ARM → NVIC

↳ Nested vector Interrupt
Control

* Vector Interrupt

3 Conditions (outside)

Processor Manager

① Global Interrupt Enable (GIE)

② Peripheral Interrupt Enable (PIE)

③ Finish Event → set Flag

H.W work

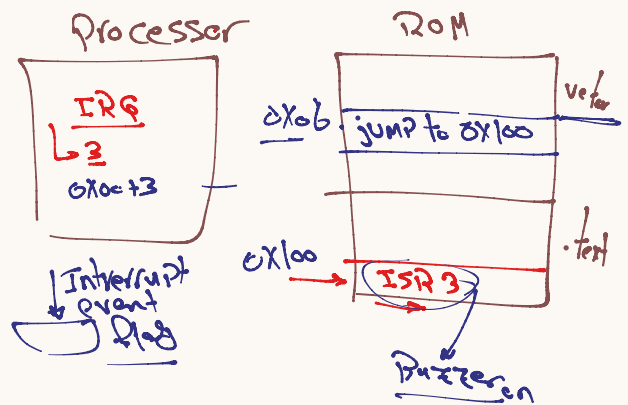
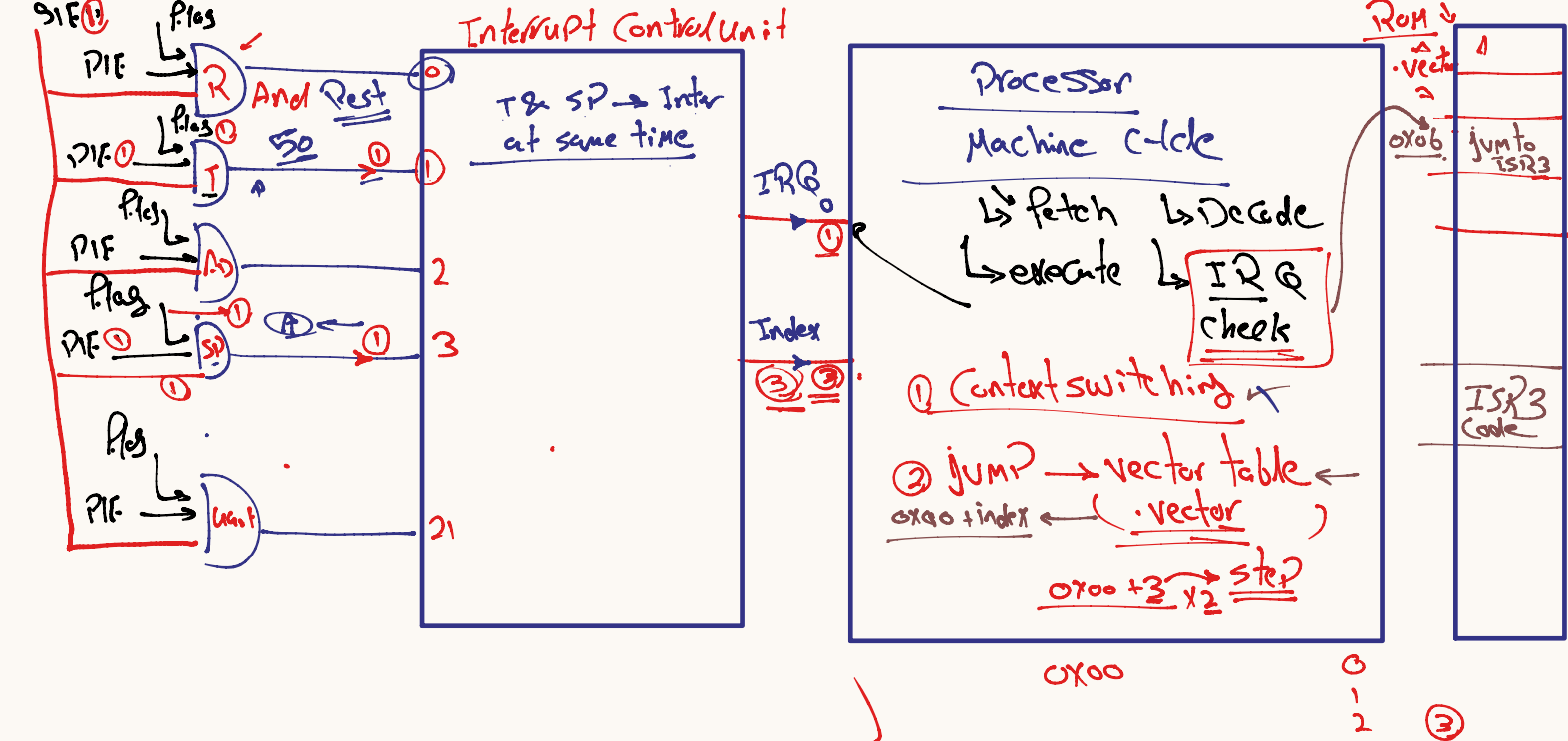
ADIE → ADC
UARTIF →

GIE ✓
PIE ✓
Flag ✓

Processor

ISR

Interrupt
Service
Routine



* What Happen when Interrupt triggered?

① Processor complete of current instruction. (H.W)

→ Machine cycle → Fetch | Decode | execute | check IRQ

② Disable Global Interrupt ⇒ (H.W)

→ not support → nested Interrupt

③ Save Program Counter (PC) in stack ... (H.W)

④ save CPU Registers (GPR, SP, Accumulator, Psw (SREG))
in stack (SW)

⑤ jump to vector table → (SW)

⑥ jump to ISR to execute (SW)
→ clear flag

- 7] Enable Global Interrupt
- 8] get the CPU Register
- 9] Set the PC
- 10] next machine cycle (fetch)

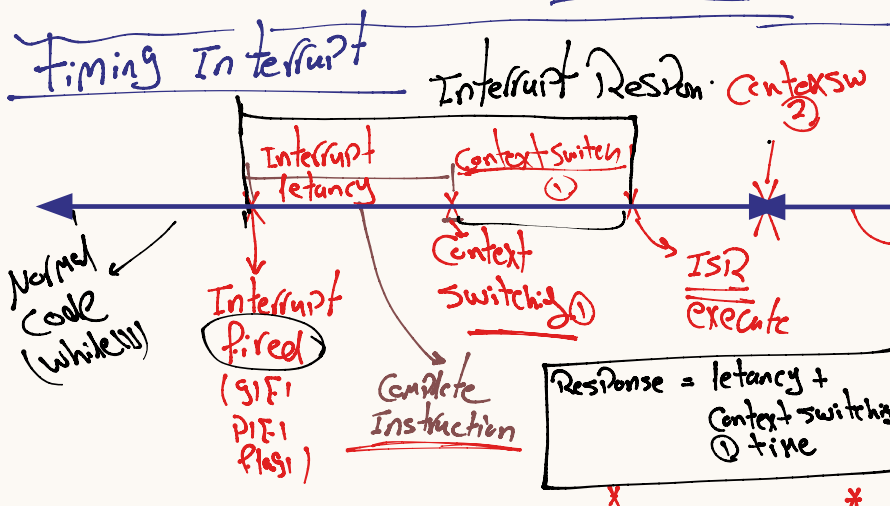
* Context switching

- 1] store the CPU Register to stack (PC, SP, SPAR)
- 2] and get the CPU Register from stack

* Context switching type

- 1] H.W context switching
 - ↳ Interrupt by H.W
 - ↳ AVR
- 2] SW context switching
 - ↳ function
 - ↳ bit assembly

Timing Interrupt



Interrupt Response → time

time between → Interrupt fired and start execute ISR

Interrupt latency

time between Interrupt fired & start first context switching

* Flag →

1] will be set after finish the work by H.W even if you use Interrupt or not → #

2] clearing

Auto clear

clear bit H.W
if you used Interrupt when start the exec of ISR

Manual clear

clear bit SW
if you not used Interrupt (Polling) check flag if (flag == 1)

Manual clear flag

1] clear flag bit zero

2] clear flag bit one → (AVR)

Read flag memory



0 → O.C
1 → S.C

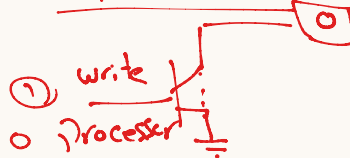
Normal Code
flag → set

Peripheral

trans

AVR

Read



clear flag bit zero

Process

Read

Write Processor

*Polling

- ↳ Peripheral → Flag up
- ↳ Processor → ask Periodic on Flag
- ↳ Manual Reset

Interrupt

- ↳ 3 Condition → ✓
- ↳ Processor Sense Interrupt
- ↳ jump to ISR execute
- ↳ Auto clear bit H.W per Flag

ISR → H.W Function

Prototype → Implementation

Call bit H.W → No Call bit SW

ISR → for AVR :: for each Interrupt specific ISR

ISR → ADC

ISR → Tx UART

ISR → Rx UART

Timer → out → ISR
CTC → ISR

ISR - write Function

Prototype

void __ vector_Index (void)

void __ vector_1 (void)

--attribute-- ((signal));

#pragma

tell compiler something

H.W

Implementation

void __ vector_Index (void)

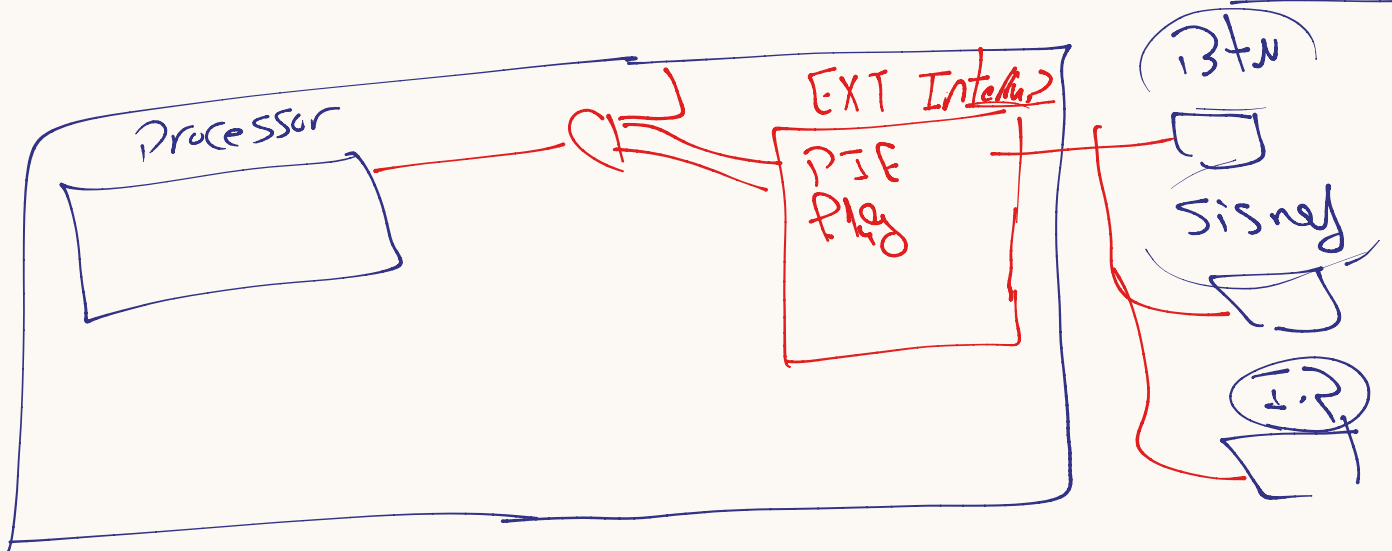
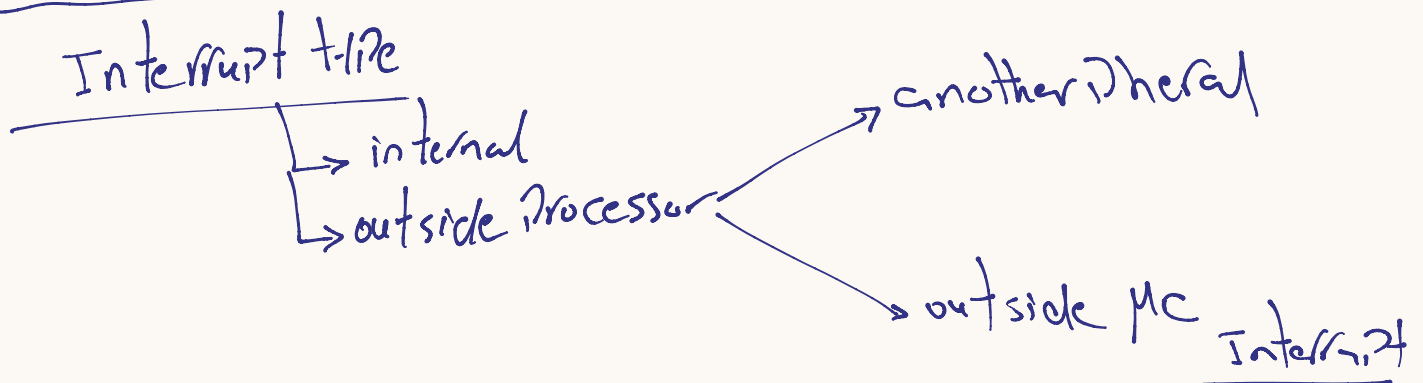
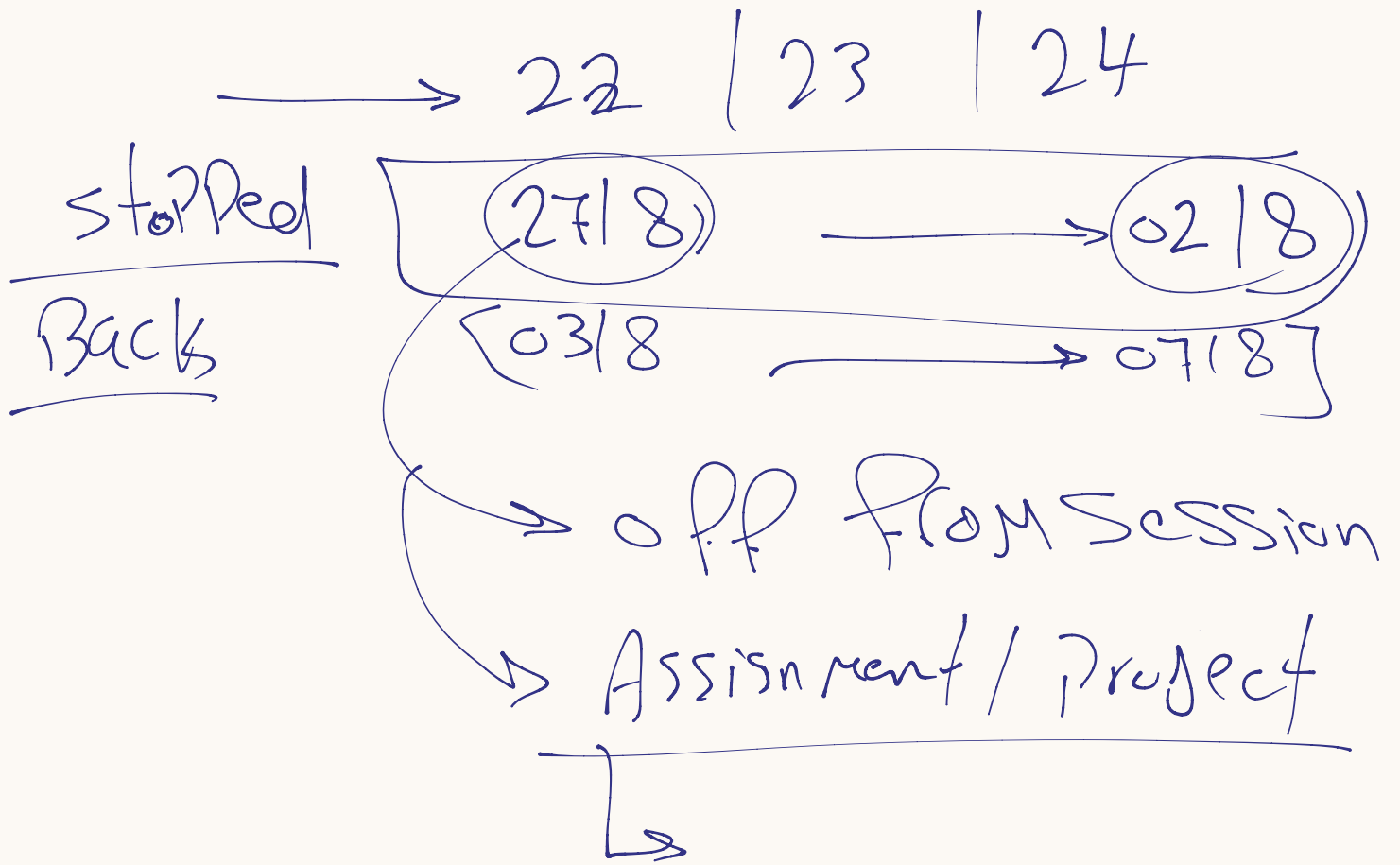
≡

ISR

void __ vector_No (void) __attribute__((signal));

void __ vector_No (void)

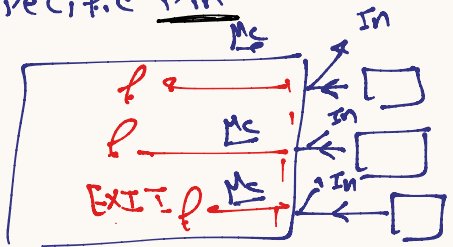
Your ISR Code below



* EXTI Interrupt

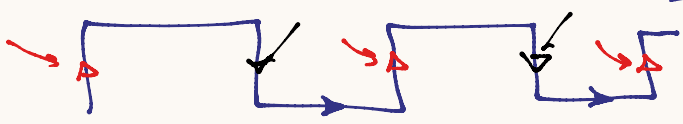
Handle the Interrupt coming outside MC

* EXTI → Peripheral → specific
specific pin

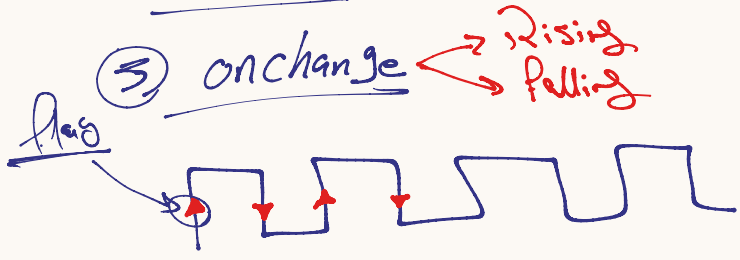


→ check input according configuration
↳ set flag

→ Configuration



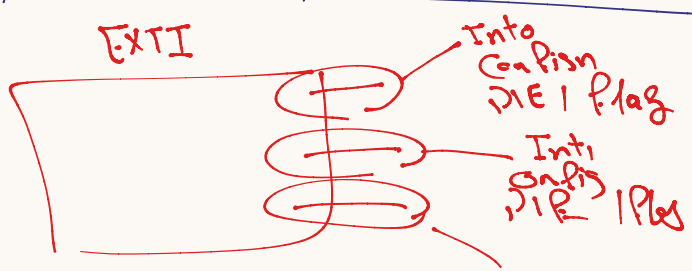
- ① Rising ② Falling
- ③ low level ④ High level



→ EXTI → check → flag
event triggered

- * EXTI →
- ① Configuration
 - ② flag
 - ③ EXTI - Enable

* Global Interrupt



PD2 → INT0 → In
PD3 → INT1 → In
PB2 → INT2 → In

Configuration ATmega32

- ↳ Low level (Int0, Int1)
- ↳ Rising (Int0, Int1, Int2)
- ↳ Falling (Int0, Int1, Int2)

MCUCR	INT1	Interrupt1
Bit No.	③	②
① Low	0	0
② onchase	1	0
③ Falling	0	1
④ Rising	1	1

if you need work Int1
as Falling

→ Clear Bit(MCUCR, 3);
→ Set Bit(MCUCR, 2);

MCUCR	Interrupt0
①	⑥
Low	0
onchase	1
Falling	0
Rising	1

↳ Code → onchase
set Bit(MCUCR, 1)
clear Bit(MCUCR, 0)

Code → low level
clear Bit(MCUCR, 1);
clear Bit(MCUCR, 0);

MCUCSR

→ Bit 6 → 0 Falling
 → 1 rising

GICR → 3-bit
7, 6, 5

7 → INT1
 ↳ PIF of exti 1
 ↳ enable
 #define INT1 7
 setBit(GICR, INT1);

⑥ → INTO
 ↳ PIF of exti 0
 ↳ enable
 #define INTO ⑥
 setBit(GICR, INTO);

⑤ → INT2
 ↳ PIF of exti 2
 ↳ enable
 #define INT2 ⑤
 setBit(GICR, INT2);

Exti Drive

 ↳ Program.c
 ↳ Interface.h
 ↳ Config.h
 ↳ Private